CS482 Software Project Proposal: Football Team Management

Nyrique' Butler, Andrew Dominquez, Leslie Kim, Chloe Miranda, Jack Wehberg

2025-11-09

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Cui
- Client title: stockholder
- Client email address: lcui@loyola.edu
- Client employer: Kids' Activity Management Organization
- How you know the client: Our client, Dr. Cui, is acting as a stockholder for a teens soccer league.

2 Project Description

2.1 Overview

Our client, Dr. Cui is a main stockholder of Kids' Activity Management Organization. He is requesting a web application specifically for a local soccer league to allow league administrators to manage seasons and for the adults registered in the league to have a place to view game schedules for the whole season. This will also be used as a platform for registered adults in the league to connect with each other online as a community.

2.2 Key Features

Key features:

- Login
- calendar of matches that spans for the season
- match generator at start of season
- match manager as season progresses into playoffs

social media

- picture/video feed
- commenting on posts
- liking posts

2.3 Why this Project is Interesting

This app centralizes access to the league not only for registered parents but for prospective parents who are looking to enroll their kids in an activity. This app is also enticing because it uses database management and a social media style interface on its main page. For example, user accounts are stored in the database, the admin can update the scoreboard and playoff bracket, and registered adults can post comments, videos, or images on the feed.

Additionally, this project is meaningful because it makes it easier for parents and their kids to stay informed about matches, team performance, and schedule. It is also interactive for registered parents to share media and engage with others. The app supports the league community by providing an online space to connect and stay informed throughout the season.

2.4 Areas of CS required

The most relevant subfields are web development and database management.

2.5 Potential Concerns and Questions

All of us have limited to no experience in web programming. We anticipate that implementing the back-end with MongoDB and Node.js will be the most difficult part. So we might have questions on how to set up the brackets or the scoreboard later on in this project.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
NFR1	Blue theme	Usability	Website should have a blue color theme
NFR2	Include logo	Usability	A custom logo should be present in the website
NFR3	Color-coded calendar	Usability	The calendar should have color-coded dates
NFR4	Post with minor protection	Security	All posted content with minors should be hidden for unregistered users
NFR5	Password security	Security	Accounts should be encouraged to have strong password
NFR6	Admin control	Security	Only the admin should handle any sort of live data
NFR7	Responsibility disclaimer	Permission	The adult should see and accept a responsibility disclaimer when registering a minor.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

ID	Story Title	Points	Description
A1	Minor Registration	1	As a/an Adult, I want to register my kid, so that my kid can participate in teens sports league.
L2	Team Manager Assignment	1	As a/an League Admin, I want to assign team manager roles to some adults, so that team manager can manage their team.
M3	Team Creation	1	As a/an Team Manager, I want to create a new team, so that I can organize players in my team.
M5	Invite Minors	1	As a/an Team Manager, I want to send invitations to registered minors to join my team, so that I can build my team.

ID	Story Title	Points	Description
M6	Manage a Team	2	As a/an Team Manager, I want to be able to add or remove team players from my team, so that I can edit my team roster and finalize my team before my season starts.
L7	Minor Reassignment	3	As a/an League Admin, I want to be the only one able to reassign minors after the season begins, so that rosters are properly controlled.
A8	Calendar Tags	1	As a/an Admin, I want to tag events in the calendar as a match, practice, or misc, so that viewers can distinguish between event types.
L9	Calendar Display	2	As a/an League Admin, I want to have a calendar display, so that I can keep track of the dates and times of games.
A10	Calendar Location	1	As a/an Adult, I want the calendar to show the location of the games, so that I can quickly get directions.
A11	Update Calendar Events	2	As a/an Admin, I want to update events in the calendar, so that I can revise event information as needed.
M12	Practice Days	1	As a/an Team Manager, I want the calendar to allow me to add practice sessions, so that parents know when we have practice.
A13	Notifications	2	As a/an Adult, I want the calendar to send me notifications, so that I can be reminded when games are happening.
A14	Different Event Forms	1	As a/an Admin, I want different forms for creating different types of events, so that the correct information is collected.
L15	Picture and Video Feed	2	As a/an League Admin, I want a feed of pictures and videos from participating adults, so that moments can be shared in the league community.
A16	Comments on Feed	2	As a/an Adult, I want to comment on posts in the feed, so that I can interact with others.
L17	Live Match Update	5	As a/an League Admin, I want a live match display that updates in real-time, so that those who cannot attend can follow the game.
A18	Match History	1	As a/an Adult, I want to view moves from old matches, so that I can help my child learn.
L19	Live Update Input	1	As a/an League Admin, I want to input moves in real-time, so that the live match display stays updated.
A20	Upload Posts	2	As a/an Adult, I want to upload photos or videos, so that I can share my experiences.
E21	Scoreboard (Live)	1	As a/an Guest, Player, Adult, Team Manager, or League Admin, I want to see live minute-by-minute match events, so that I don't miss anything.
E22	Future Matches	1	As a/an user, I want to view future match details, so that I can prepare.
E23	Past Matches	1	As a/an user, I want to view past match details, so that I can analyze and learn.
R24	Post Reaction	2	As a/an registered user, I want to react to posts on the social feed using simple reactions (happy, neutral, angry, sad, like, dislike), so that I can express how I feel on that social feed post.
A25	Add Match	1	As a/an Admin, I want to create a match, so that I can record matches.
E26	Weather	1	As a/an user, I want to know the weather, so that I am prepared.
L27	Adult Approval	1	As a/an League Admin, I want to approve adults registering, so that only valid adults register minors.
L28	Add New Admin	1	As a/an League Admin, I want to add new admins, so that administration duties can be shared.

ID	Story Title	Points	Description
L29	Calendar Matches	2	As a/an League Admin, I want to add or delete matches on the calendar, so that the schedule is accurate.
L30	Season Start Bracket	1	As a/an League Admin, I want to generate a random bracket, so that season matches are assigned fairly.
L31	Automated Match Decisions	1	As a/an League Admin, I want matches to be automatically generated based on results, so that match progression is fair.
L32	Playoff Generation	1	As a/an League Admin, I want playoff matches generated based on season data, so that playoffs are fair.
L33	Championship	1	As a/an League Admin, I want the championship to be generated automatically, so that the season concludes fairly.
G34	Account Creation	2	As a/an Guest, I want to create an account, so that I can use site features.
E35	User Login/Logout	1	As a/an user, I want to log in and out securely, so that my information is safe.
L36	Admin Dashboard	1	As a/an League Admin, I want only admins to access the admin dashboard, so that admin tasks are restricted.
L37	Delete Match	1	As a/an Admin, I want to delete a match, so that I can adjust schedules.
L38	Update Match	1	As a/an Admin, I want to update match details, so that schedules remain accurate.
A39	Team Invitation	1	As a/an Adult, I want to approve or deny team invitations for my child, so that I control their participation.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

Our team is following the MVC architecture. In our design, the "View" layer would handle the user interface with HTML, CSS, and JavaScript. The "Model" layer would handle the database using MongoDB. The "Controller" layer would manage incoming requests by using Node.js features. This includes applying user account permission and account authentication. We will use the Express module to handle requests.

4.2 Diagrams

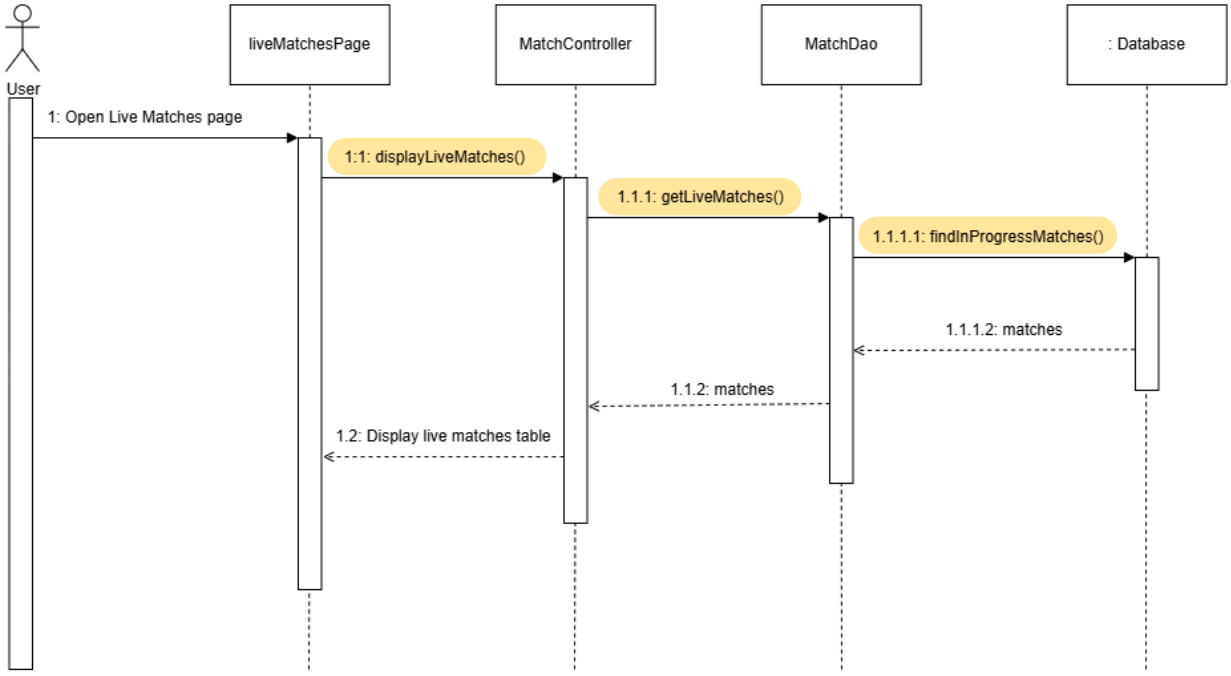

Figure 1: Iteration 2 - Live Matches Sequence Diagram

4.3 Technology

The programming language our team will be using is JavaScript, along with HTML & CSS for webpage development. JavaScript allows us to use the same language for both client and server side, making development more efficient. We will be using Node.js as our main framework as it allows us to use JavaScript on the server side as well. Our team will be using MongoDB as our database because it integrates easily with JavaScript. To test our code, we will be using Jest as it is simple to write & run tests, helping us to meet our coverage goal.

4.4 Coding Standards

The team will follow consistent coding standards to ensure readability, maintainability, and reliability throughout development. Only working code will be committed to the repository, and each commit message must clearly describe what was changed or added. Code will be pushed in small increments to make testing and debugging easier, and any commented-out or developer-only notes will be removed before release. All variable declarations will use const or let, instead of var, and complex methods will include inline comments explaining their purpose. File names will follow the lowercase-hyphen format, variables and functions will use camelCase, and constants will be written in UPPERCASE_WITH_UNDERSCORES. Test names will clearly describe the function being tested and the expected outcome.

Functions will be kept short, serving a single clear purpose, and each will include JSDoc-style comments explaining its parameters and return values. Code formatting will follow a consistent style, including two-space indentation, single quotes for strings (unless otherwise necessary), one blank line between functions, and removal of unused variables or imports. Testing will be conducted using Jest, with at least 60% code coverage required for commits and a final goal of 80% coverage at project completion. Project documentation will be maintained alongside code updates to ensure accuracy and alignment with the current implementation.

4.5 Data

Here is a description of the entities that will be stored in our database for the football league.

The football league is comprised of a league administrator, teams, team managers, registered adults and their registered kids. A league administrator or "admin" has a unique id and their name, username, email, phone number, date of birth and login password stored. A league is comprised of teams. A team has a unique id, a team manager, team name, and a roster of registered kids. Team managers have a unique id, their name, username, email, phone, date of birth, login password and their team's id. Registered kids are registered by their parents and they input their children's name, and date of birth. A kid cannot register itself. Parents/guardians of kids must be registered adults. A registered adult has their name, username, email, phone number, date of birth and login password stored as well as the id of their registered kid.

In the league, there is a season for football. A season has a unique id, a start date, an end date and an eventual champion. Seasons are comprised of games. A game has a unique id, date, start time, end time, place, weather, status in the season (normal, playoff, championship), a winning team and a losing team.

Live chats are recorded on the landing page during games and are comprised of messsages. Messages have a unique ID, a username from registered adult, date and time.

resulting tables

- League_Admin(@id_admin, name, email, phone, date_of_birth, password, username)
- Team_Manager(@id_manager, name, email, phone, date_of_birth, password, #id_team, username)
- Registered_Adult(@id_adult, name, email, phone, date_of_birth, password, username)
- Registered_Kid(@id_kid, name, date_of_birth, #id_adult, #id_team)
- Team(@id_team, name_team, #id_manager, wins, losses)
- Season(@id_season, start_date, end_date, #id_champion_team)
- Match(@id_game, date, start_time, end_time, final_score, place, weather, status, #id_team1, #id_team2, #id_season)

4.6 UI Mocks

The entire sketch so far is hypothetical and is subject to change upon testing or further discussion as to what should be prioritized upon first loading the website. When the page is loaded, a guest/user/admin will see the home page that has a feed centered in the center with an active standings tab to the left, a scaled down schedule display on the right and another widget just below that. There would be a static bar at the topic that will prompt users to log in at the top right. Once a user presses log in, there would be an option to enter your email and password. Otherwise, a guest will create a new account via a sign up button that will appear at the bottom of the card. The sign up card would ask for a whole bunch of information that will serve as an identifier to ensure not just anybody can sign up for the website. It was originally planned for the administrator dashboard to be its on page, but upon discussion it was decided that the administrator dashboard would be appear on the profile of the admin and be hidden on all non-administrator accounts.

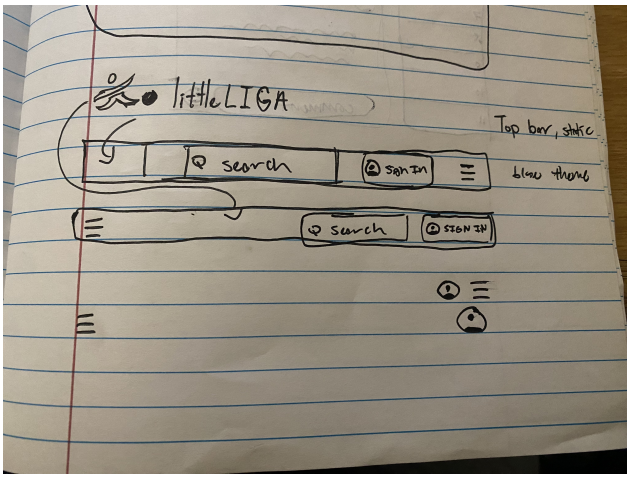

Figure 2: Static Bar

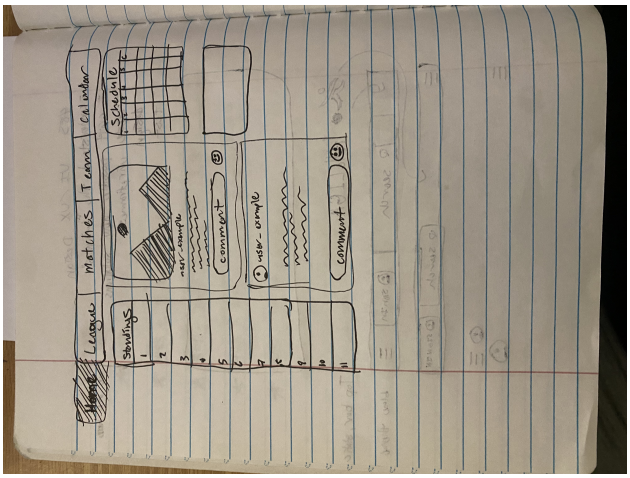

Figure 3: Main landing page

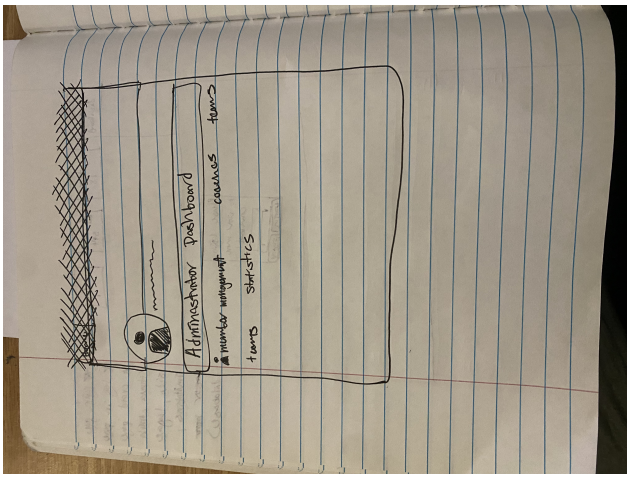

Figure 4: very rough Admin dashboard on profile page

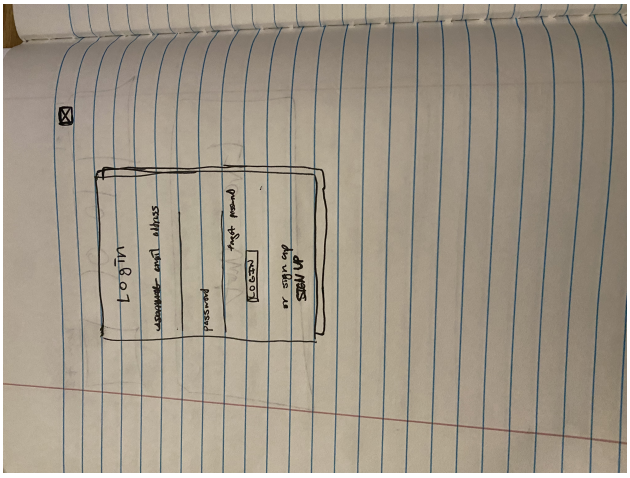

Figure 5: part 1 of log in page(login)

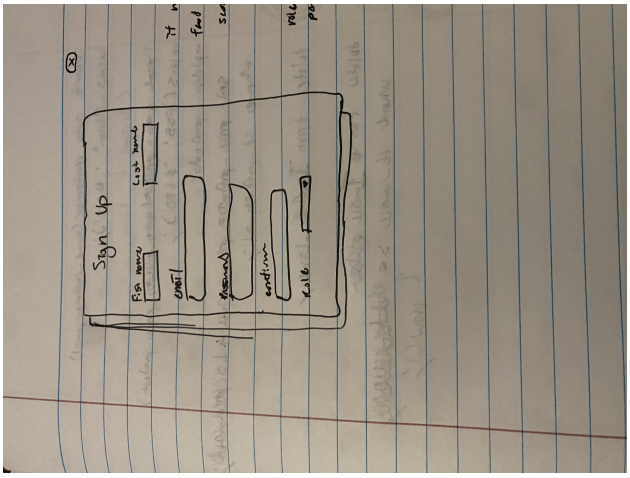

Figure 6: part 2 of log in page(sign up)

5 Iterations

5.1 Iteration Planning

Iteration	Dates	Stories	Points
1	10/09 - 10/23	G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment, L27 Approving adults, L9 Calendar display, L29 adding/deleting matches from calendar, M6 Managing the team, M3 creating a new team, M5 inviting minors to teams, L30 creating randomized bracket at start of season, L31 automatic bracket generation throughout season, L32 playoff bracket generation, L33 Championship/season ending, E21 scoreboard (live), E22 future matches, E23 past matches, E26 weather	24
2	10/23 - 11/06	E21 scoreboard (live), E22 future matches, E23 past matches, E26 weather, M6 Managing the team, M3 creating a new team, M5 inviting minors to teams, A39 Team Invitation, L32 playoff bracket generation, L33 Championship/season ending, M12 Practice Days, A10 Calendar location, L7 Minor reassignment, A20 Upload posts, L15 Picture and Video feed, A8 Calendar tags, A14 different event forms, A11 Update calendar, L25 Add Match, L37 Delete Match, L38 Update Match, T4 Spike work for A20	26
3	11/06 - 11/20	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21
		Total:	70

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

The stories we initially planned for this iteration are listed on the table at the Iteration 1 row, excluding G34 Account Creation, E35 User Login/Logout, and L36 Admin Dashboard. Our focus for the first iteration is to choose some user stories from most of the major features, which are admin, manager, calendar, bracket generation, and match data, to get started. It was decided that Chloe would do the calendar, Leslie would do admin, Jack would do manager, Andrew would do bracket generation, and Nyrique would do match data features.

At the beginning, we planned for everyone to do four story points. But we forgot about account features and the admin dashboard, which are prerequisites to most of our planned user stories. So, G34 Account Creation, E35 User Login/Logout, and L36 Admin Dashboard were added after planning and during the iteration because they were high priority for the admin feature user stories and setting up the website. Because of it, one member ended up doing more story points than others unintentionally.

5.2.2 Work Done

The stories we completed are G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment, L27 Approving adults, L9 Calendar display, L29 adding and deleting matches from the calendar, M3 Team creation, M5 inviting minors, M6 managing a team, and L30 season start bracket.

The stories we partially completed are E21 live scoreboard, E22 future matches, E23 past matches, E26 weather, L31 automated match decisions, L32 playoff generation, L33 Championship

Chloe worked on L9 Calendar display and L29 adding/deleting matches from calendar. Leslie worked on G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment and L27 Approving adults. Jack worked on M6 Managing the team, M3 creating a new team, and M5 inviting minors to teams. Andrew worked on L30 creating randomized bracket at start of season, L31 automatic bracket generation throughout season, L32 playoff bracket generation, and L33 Championship/season ending. Nyrique worked on E21 scoreboard (live), E22 future matches, E23 past matches, and E26 weather.

5.2.3 Testing Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.59	84.74	96.72	93.81	
FootballTeamManagement	100	100	100	100	
CalendarConfig.js	100	100	100	100	
FootballTeamManagement/controller	95.89	100	100	95.89	
EventController.js	100	100	100	100	
InviteMinor.js	100	100	100	100	
MinorController.js	90.47	100	100	90.47	28,42
TeamCreation.js	100	100	100	100	
TeamManagement.js	100	100	100	100	
UserController.js	90	100	100	90	17,48,85,99
FootballTeamManagement/model	88.46	50	94.73	89.21	
DbConnection.js	90.9	50	100	100	10-11
EventDao.js	100	100	100	100	
MatchupDao.js	100	100	100	100	
MinorDao.js	100	100	100	100	
SeasonScheduleDao.js	75	50	85.71	75	49-50,123,131,147,160-167
TeamDao.js	100	100	100	100	
UserDao.js	100	100	100	100	
FootballTeamManagement/util	100	100	100	100	
PasswordUtil.js	100	100	100	100	

Figure 7: Iteration 1 coverage report

This coverage is good enough because all tests statement coverage are above 80%. We tested most parts of the code for the Controller and the Model. We didn't test the "catch (err)" block in the try-catch that most functions have because the coverage already shows good enough statement coverage and it seems redundant to test if it catches the error properly. For Calendar's testing, Chloe got some weird edge cases like leap years but there may be more found later, and coverage may not show this.

5.2.4 Retroespective & Reflection

The first challenge in this iteration was setting up the programming environment since it was originally confusing to understand how to connect MongoDB to the program and the purpose of the App.js and Server.js from the example code. After referencing some tutorials and somewhat referring to "ParkingLotExampleJS", we learned how to set up everything properly.

Also related to the programming environment, there was an issue in MongoDB database. Before the fix, the database would clear the data after a couple hours. This was an inconvenience when testing the code manually, especially testing admin features. The solution was to not use the default database, test, by creating a new database on the website and edit the ".env" file to include that database's name.

On the topic of databases, the user stories we worked on for this iteration had some things that relied on others, and because we worked on those components separately, we had to ensure that we knew how data was going to be sent to the server.

Another major challenge was creating the Jest test code for the Controller Javascript files. It was extremely confusing at first because the entire example tests for the Controller in "ParkingLotExampleJS" didn't make sense. But it became clearer once we researched about Jest mock functions and noticed how the Jest test code lines corresponded to the code being tested.

Styling the frontend was also challenging. For example, we wanted to use a Bootstrap modal for team ID input in minor reassignment. Implementing the modal didn't go as planned because we couldn't get it to work. In the end, using prompt() was sufficient, even though it was not as visually appealing as a modal.

Lastly, there was an issue in planning, which we discovered during the iteration, because we forgot to include user stories for account features and admin dashboard in our spreadsheet, which are connected to most planned stories in this first iteration.

To summarize, in this iteration, we learned how to set up MongoDB and use Node.js with Express to implement the Controller and Model. It was also our first time working with backend, which helped us understand how the database interacts with the server through routes.

For our next iteration, it will probably be best to have better planning for our user stories so that they are a little bit more unified and make an order that shows logical progress rather than grab at available points.

5.3 Iteration/Sprint 2

5.3.1 Planning

The stories we planned for this iteration are listed in the Iteration 2 row of the table. From the three new epics we added, which are Calendar, Social Feed, and Live Matches, to our User Story spreadsheet, we decided to begin with the Calendar and Social Feed epics because Calendar is already in progress and Social Feed is independent of the Calendar epic. The purpose of these epics is to stay organized by grouping user stories related to each major feature, which is high priority. We selected user stories that are more logical next step and unified so that we could plan them more effectively this time.

The user stories from the previous iteration are E21 scoreboard (live), E22 future matches, E23 past matches, E26 weather, L7 Minor reassignment, M6 Managing the team user stories, M3 creating a new team, M5 inviting minors to teams, L32 playoff bracket generation, and L33 Championship/season ending. E21, E22, E23, and E26 weren't completed previously. L7 user story's point was adjusted from 3 to 1 point left for this iteration according to feedback. M6 user story's point was also adjusted from 2 to 1 point left for this iteration according to feedback.

5.3.2 Work Done

The stories we completed are L7 minor reassignment, T4 Spike work for A20, A20 Upload posts, L15 Picture and Video feed, L25 Add Match, L37 Delete Match, L38 Update Match, A11 Update calendar, A8 Calendar tags, L32 Playoff Bracket Generation, and L33 Championship/season ending. Based on the completed user stories, the total points we did is 16 points.

The stories we partially completed are M6 Managing the team, M3 creating a new team, M5 inviting minors to teams, A39 Team Invitation, M12 Practice Days, A10 Calendar location, E21 scoreboard (live), E22 future matches, E23 past matches, and E26 weather.

Chloe worked on A8 Calendar tags, A14 different event forms, A11 Update calendar, L25 Add Match, L37 Delete Match, and L38 Update Match. Leslie worked on L7 Minor reassignment, A20 Upload posts, L15 Picture and Video feed, and T4 Spike work for A20. Jack worked on M6 Managing the team, M3 creating a new team, M5 inviting minors to teams, and A39 Team Invitation. Andrew worked on L32 playoff bracket generation, L33 Championship/season ending, M12 Practice Days, and A10 Calendar location. Nyrique worked on E21 scoreboard (live), E22 future matches, E23 past matches, and E26 weather.

From Leslie's user stories, L7 minor reassignment took about 0.5 hour, the T4 Spike work for A20 took 2 hours, A20 Upload posts took 2 hours, and L15 Picture and Video feed took 4-5 hours. L15 took longer than expected because of having to debug mismatched div tags and many unnoticed typos in the HTML code.

From Chloe's user stories, L25, L37, and L38 (Create updating and deleting a match) was about maybe 4-6 hours combined. A11 was about two hours, and A8 and A14 was maybe about 2-4 hours, possibly more after planning and reconfiguring a few things. From Andrew's user stories, L32 and L33 took roughly 3-4 hours. But it is unknown for Nyrique's and Jack's work hours.

5.3.3 Testing Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.59	80.86	96.39	91.6	
FootballTeamManagement	100	100	100	100	
CalendarConfig.js	100	100	100	100	
FootballTeamManagement/controller	91.06	93.05	97.67	90.98	
EventController.js	100	100	100	100	
InviteMinor.js	100	100	100	100	
MatchController.js	100	100	100	100	
MinorController.js	90.47	100	100	90.47	28,42
PlayoffController.js	81.25	50	100	81.25	13,28,32,47,63,79,83,98,112
PostController.js	100	100	100	100	
ScheduleController.js	68.75	25	85.71	68.75	13,28,32,47,63,78,89-98,112
TeamController.js	85.71	100	100	85.71	14
TeamCreation.js	100	100	100	100	
TeamManagement.js	100	100	100	100	
UserController.js	90	100	100	90	17,48,85,99
FootballTeamManagement/model	88	50	95.23	91.51	
DbConnection.js	90.9	50	100	100	10-11
EventDao.js	100	100	100	100	
MatchDao.js	100	100	100	100	
MatchupDao.js	100	100	100	100	
MinorDao.js	100	100	100	100	
PlayoffBracketDao.js	81.81	50	100	94.44	68-69
PostDao.js	91.66	100	83.33	91.66	44
SeasonScheduleDao.js	75	50	85.71	75	46-47,120,128,144,157-164
TeamDao.js	100	100	100	100	
UserDao.js	100	100	100	100	
FootballTeamManagement/util	100	100	100	100	
PasswordUtil.js	100	100	100	100	

Figure 8: Iteration 2 coverage report

This coverage is good enough because the average code coverage is ~~above 80%~~ and the average branch coverage is about 50% as shown in 'All files' row. There is no test for PostDao's readAll function because testing the .populate(...).sort(...) call unexpectedly caused one of the UserDao tests to fail when the test coverage was run more than once. Even so, PostDao's statement coverage is still above 80%. Currently, Chloe is still trying to figure out how to thoroughly test the frontend of Calendar without reconfiguring everyone else's tests, since it requires a different testing software. Regardless Calendar's backend tests (EventDao, EventController, CalendarConfig) are all 100% and pass.

5.3.4 Retroespective & Reflection

One challenge during this iteration was coding the UI. We realized that after coding large portions of HTML at once, small errors, such as typos that don't match the back-end's variables, can go unnoticed and become harder to trace later. This caused debugging to take longer than expected. To improve in the next iteration, we plan to take a more incremental approach to UI development, such as implementing and testing one input at a time. This will allow us to identify issues earlier and reduce time spent debugging the front-end.

Another challenge was the amount of refactoring of the UI that had to happen to implement small features. For calendar tags, most of the work was the front end and I had to do a lot more for that than I did for updating a calendar event. There were also more user design choices that were being made when adding things to the calendar since it was mostly functional before.

Overall, we learned that working with the UI is more challenging than we initially expected, particularly because of the design decisions involved. Unlike JavaScript, HTML does not clearly indicate where errors

occur, making debugging the front-end more time consuming and less straightforward. Additionally, we learned how to use the Multer middleware to upload a single media file.

For our next iteration, it will be important to plan the UI more intentionally before coding. This can mean sketching out the design or listing the necessary UI elements before coding. Additionally, taking a more incremental development approach on the front-end and testing one UI component at a time rather than everything at once will help us catch issues earlier and reduce time spent refactoring later.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]